Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 6

Users, Groups and File Permissions

Lab 6 guide: read the mode bits, set the least access that works, and follow the whole path to a file.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Read owner, group and other mode bits, in symbolic and octal notation.
2. Set least-privilege modes on files and directories you own, and explain directory traversal.
3. Diagnose an access failure from path and mode evidence, and separate what you observed from what you only reasoned.

THIS LAB

Lab 6

First the concepts, with examples. Then the lab on the server, using only files you own. Accounts, ACLs and special bits are optional extensions, so they come last.

DISCUSSION

A file is readable by everyone. Can everyone read it?

What if the folder that holds the file is private? Where does the check really start?

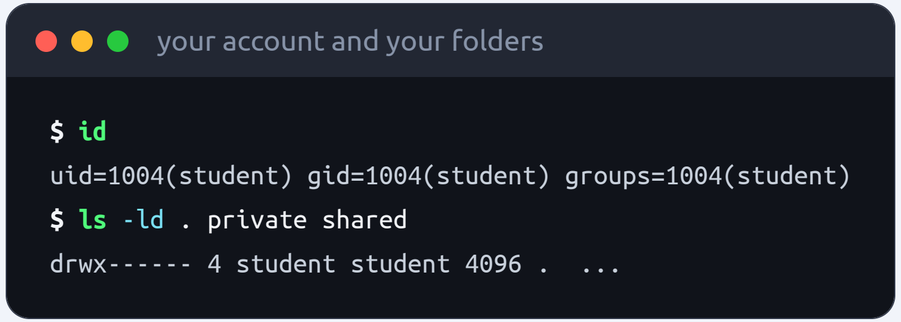
CONCEPTS FIRST

Who may do what to a file

Three ideas: three kinds of user, three kinds of access, and a check on every folder of the path.

Users and groups

- Every file has an owner (a user) and an owning group.
- Every process runs as a user, and has a primary group and sometimes more.
- The kernel compares who is asking with the owner, the group and everybody else, in that order.
- The first match decides. A normal user cannot give a file to another account.



Reading the mode string

student@server: ~/oslab-work/lab6

```
$ ls -l report.txt
-rw-r----- 1 student staff 120 report.txt
```

Characters	Part	Here
`-`	Type: - file, d directory, l link	A plain file
`rw-`	Owner	Read and write
`r--`	Owning group	Read only
`---`	Everybody else	No access

The first character is the type. The next nine are three triples: owner, group, others. A dash means that access is not given.

Octal and symbolic modes

Bit	Value	On a file	On a directory
r	4	Read the contents	List the names inside
w	2	Change the contents	Create, rename or remove entries
x	1	Run it as a program	Search: pass through to what is inside

Add the values of one triple to get one digit. rw- is 4+2 = 6, r-- is 4. So rw-r----- is 640.

chmod, in two notations

student@server: ~/oslab-work/lab6

```
$ chmod 600 note.txt
$ chmod u=rw,g=r,o= note.txt
$ stat -c '%A %a %n' note.txt
-rw-r----- 640 note.txt
$ chmod g-r note.txt
$ chmod u+x script.sh
```

Octal sets all nine bits at once. Symbolic mode changes only what you name: u owner, g group, o others, = sets, + adds, - removes.

stat -c shows the mode in both forms: %A symbolic, %a octal.

On a directory, x means search

READ

r on a directory

You may list the names inside it, with ls.

SEARCH

x on a directory

You may pass through it to reach an entry whose name you already know. Without x you cannot open anything below it.

TOGETHER

Both are needed to browse

To list names and open files you need r and x. A directory with only r shows names but opens nothing.

Do not try to run a data file to test read permission. For a directory, x is not about running a program.

The whole path is checked

student@server: ~/oslab-work/lab6

```
$ namei -l "$PWD/demo/note.txt"
f: /home/student/oslab-work/lab6/demo/note.txt
drwxr-xr-x root    root    /
drwxr-xr-x root    root    home
drwx------ student student student
drwx------ student student oslab-work
-rw-r----- student student note.txt
```

To open a file you need x on every directory of the path, from / down to the file.

A file with mode 644 inside a 700 directory is out of reach for other users. The strictest folder wins.

If namei is missing, list each parent with ls -ld.

LAB 6

A private record and a public notice

Configure your own copy first, and explain the access path before anyone prepares a multi-user exercise.

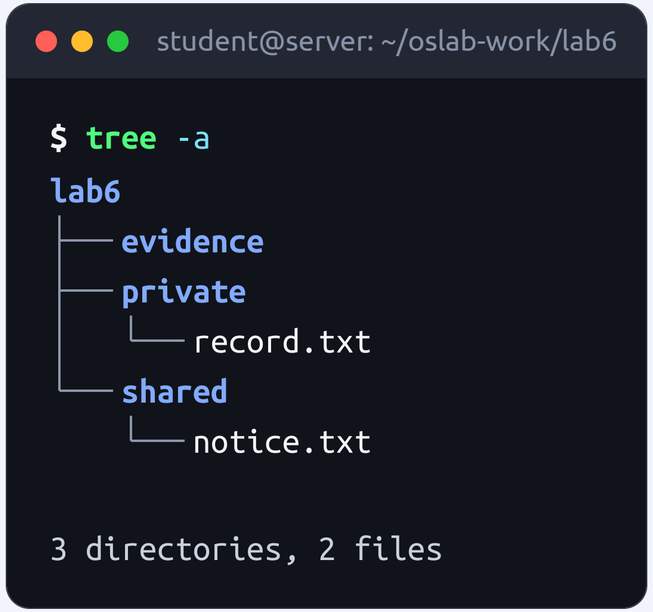
The scenario

SCENARIO

Records at TechCorp

TechCorp needs a private record and a notice that staff can eventually read. Configure your owned copy first and justify the whole access path.

Your notice policy and your demo folder name are your own. This guide shows example values.



Inside the 120 minutes

Minutes	What happens
0 to 10	Setup
10 to 25	Guided: permission basics
25 to 35	Prediction
35 to 55	Core 1: set the modes
55 to 70	Core 2: prove it

Minutes	What happens
70 to 85	Instructor demonstration, Plus and Challenge
85 to 100	Live checkpoint
100 to 110	Debrief
110 to 120	Submit

Six helper commands, and why you use them

Command	What it does
`oslab values lab6`	Shows your notice policy (a sentence) and the name of your demo folder
`oslab predict lab6`	Saves your prediction once, before the experiment
`oslab hint lab6 1`	A hint, levels 1 to 3. Hints cost no points
`oslab check lab6`	Reads the modes of your files and shows your milestones
`oslab checkpoint lab6`	Opens when the instructor says so, near the end
`oslab debrief lab6`	After the lab: you give the right answers to your prediction questions

Before the lab, run oslab doctor and oslab prelab lab6. It takes about ten minutes.

How you work

Core, Plus, Challenge

Core is for everyone. Plus and Challenge are for students who finish early. Full marks need only Core and the checkpoint.

AI tools

Allowed in Core, Plus and Challenge, so test every suggestion with stat. Not allowed in the prediction and the checkpoint.

Only your own files

You never use sudo, never change your home folder, and never try to read a neighbour's files.

In Core 1 nobody gives you the modes. You turn your policy sentence into numbers yourself. A wrong prediction costs nothing.

Setup: start the lab

student@server: ~

```
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab start lab6
$ cd "$OSLAB_WORKSPACE/lab6"
$ mkdir -p evidence
$ oslab values lab6
```

- The first line sets OSLAB_WORKSPACE to ~/oslab-work, unless it is already set. Type it as it is.
- oslab start lab6 makes the lab folder.
- cd takes you into it, and mkdir -p makes the evidence folder.
- oslab values lab6 prints your own policy and name.

Your values: a policy and a name

The last command prints them

example: your values will be different

```
$ oslab values lab6
Your personal values for lab6 (your-account):
  notice = The owner may change the file.
    The owner's group may read it and list
    the folder. Others have no access.
  box    = sealed-maple
```

What they are for

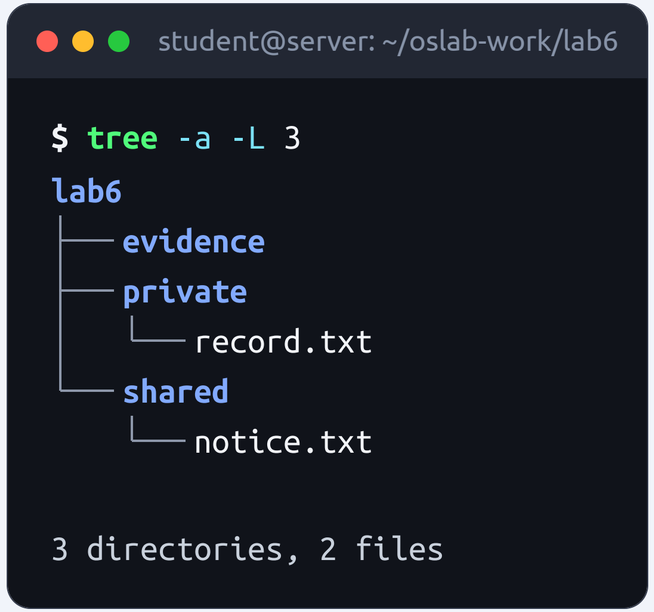
- The notice policy is the sentence you turn into modes in Core 1.
- The box value is the name of your demo folder in Core 2.
- Your own policy and name are different. Write them down.

The capital word is a placeholder

The instruction says	With the example value, you type
`mkdir -p demo/BOX`	`mkdir -p demo/sealed-maple`

The instruction cannot know your name, so it writes BOX where your box value belongs. Never type BOX. Type your own value in its place.

Your folder



private/record.txt is your private record.

shared/notice.txt is the notice that others should be able to read, once an administrator puts it in a searchable place.

oslab check lab6 shows TRY on every line at the start. The hidden .oslab-managed.json is the helper's. Leave it.

GUIDED

Permission basics

Ten to twenty-five minutes, with the instructor. Read modes, and change them on a demo file that is not part of your answer.

Step 1: who am I, and what do I own?

student@server: ~/oslab-work/lab6

```
$ id
$ ls -ld . private shared
$ stat -c '%A %a %U:%G %n' private/record.txt shared/notice.txt
```

Nothing is changed in this step. id shows your user and groups.

%A is the mode in letters, %a in octal, %U:%G the owner and the group, %n the name.

Each triple of the mode applies to the owner, the owning group and others. A directory's x bit is search, not running a program.

Step 2: octal and symbolic, on a demo file

student@server: ~/oslab-work/lab6

```
$ mkdir -p demo
$ printf 'demo content\n' > demo/note.txt
$ chmod 600 demo/note.txt
$ stat -c '%A %a %n' demo/note.txt
-rw------- 600 demo/note.txt
$ chmod u=rw,g=r,o= demo/note.txt
$ stat -c '%A %a %n' demo/note.txt
-rw-r----- 640 demo/note.txt
```

Use a separate demo file, so the lab files stay untouched.

The second mode, 640, lets the owning group read the file. It does not make any folder on the path enterable.

Step 3: look at every folder on the path

If namei is not installed, list each parent with ls -ld.

student@server: ~/oslab-work/lab6

```
$ namei -l "$PWD/demo/note.txt"
```

TRY

Observe

Does mode 644 on a file inside a 700 folder mean that other users can reach the file?

PREDICT

Guess first, then try

Before you set any mode. Ten minutes.

The prediction

student@server: ~/oslab-work/lab6

```
$ oslab predict lab6
```

It asks four questions and one sentence, about a folder with mode 600 and about your own notice policy. Answer alone, without AI.

PREDICTION

The rules

The first answer is the one that counts. A wrong guess costs nothing. You compare it with the real result in the debrief.

Set the modes

Twenty minutes, from minute 35 to 55. Turn two policies into modes, apply them to your own files, and save the result.

Two policies

POLICY 1

The private record

The same for everybody: only you may read and change private/record.txt, and nobody else may even enter private/.

POLICY 2

The notice

Your own sentence from oslab values lab6. It applies to shared/notice.txt and to the folder shared/.

Example notice policy: the owner may change the file, the owner's group may read it and list the folder, others have no access.

Step 3: your decision, four chmod commands

Write four commands yourself

- One for private/, one for private/record.txt.
- One for shared/, one for shared/notice.txt.
- Work out each mode from the table of mode bits: which digits do you need for the owner, the group and the others?
- Does a folder need the x bit for the people who may read a file inside it?

YOUR TURN

You choose the modes

Nobody gives you the numbers. Give the least access that still does the job.

Steps 4 and 5: save the result, then check

student@server: ~/oslab-work/lab6

```
$ stat -c '%A %a %U:%G %n' private private/record.txt \
$   shared shared/notice.txt | tee evidence/policy.txt
$ oslab check lab6
```

Run the stat line after your last chmod. The first two Core milestones must show PASS.

STUCK?

Hints

oslab hint lab6 1, then 2, then 3. AI is allowed here: test every suggestion with stat.

Prove it

Fifteen minutes, from minute 55 to 70. Show that the modes work, and show a failure that you can explain.

Step 1: use the record as its owner

student@server: ~/oslab-work/lab6

```
$ cat private/record.txt
$ printf 'owner update\n' >> private/record.txt
$ if test -x private/record.txt; then printf 'unexpected execute bit\n'; else printf 'not executable\n'; fi
```

Read and append as the owner. Then check that the file has no execute bit. A data file should not have one.

Do not run a data file to test read permission.

Step 2: a folder you can list but not enter

example name: use your own box value

```
$ mkdir -p demo/sealed-maple
$ printf 'inside\n' > demo/sealed-maple/item.txt
$ chmod 600 demo/sealed-maple
$ ls demo/sealed-maple | tee evidence/traversal.txt
$ cat demo/sealed-maple/item.txt 2>&1 | tee -a evidence/traversal.txt
$ chmod 700 demo/sealed-maple
$ cat demo/sealed-maple/item.txt | tee -a evidence/traversal.txt
```

The failing cat is intended. Restore the folder right after, with chmod 700.

ls shows the name because the folder has r. cat fails because it has no x. 2>&1 saves the error message in the file too.

TRY

Think

Which bit of the demo folder caused the failure?

Step 3: check your work

student@server: ~/oslab-work/lab6

```
$ oslab check lab6
```

All four Core milestones must show PASS: your private record, your notice policy, the traversal test and policy.txt.

Step 4: a hypothetical peer

student@server: ~/oslab-work/lab6

```
$ namei -l "$PWD/shared/notice.txt"
```

Could another student read shared/notice.txt under your policy? Look at every parent of the file, including oslab-work and your home folder. Do not change them to try it out.

REPORT

Write down for your report

The notice's own mode is not enough. Which parent decides? What was a real test above, and what is only your reasoning?

What each case shows, and what it cannot show

Case	Evidence	Limit
Owner access	Read and append the private record, then inspect the modes	Does not simulate another user
Search failure	Before, failed and restored reads under your demo folder	Shows traversal, not every ACL or capability effect
Peer access	A full-path explanation of the modes	Needs a prepared second account to confirm in practice

Default ACLs can change the mode of new files. Look at the final modes again after your tests.

If something does not work

Check who you are

Run whoami and id, and confirm you own the file before you change its mode.

chown is not for you

A normal user cannot give a file to another account. Never edit sudoers, never create accounts.

Read the manual

man chmod, man stat, man namei and man path_resolution explain the details.

Plus: the same mode in letters

example for 640: use the letters for your mode

```
$ chmod u=rw,g=r,o= shared/notice.txt
$ stat -c '%A %a %n' shared/notice.txt
```

Write the symbolic chmod command that gives shared/notice.txt the same mode as your numbers. Run it, then check that nothing changed.

PLUS

Answer in evidence/plus.txt

Which letters did you write, and why do they match your octal mode?

First watch the instructor show two accounts on a prepared machine: a file with mode 644 inside a folder with mode 711, then 700. Plus and Challenge are not needed for full marks.

Challenge: umask

student@server: ~/oslab-work/lab6

```
$ ( umask 077; touch demo/umask-test.txt; stat -c '%A %a %n' demo/umask-test.txt )
```

umask shows which bits a new file does not get. With umask 0022 a new file has mode 644 and a new folder 755.

CHALLENGE

Predict first

What is the mode of a file you create after umask 077? The subshell keeps your session's umask unchanged.

LIVE CHECKPOINT

Show what you can do alone

Fifteen minutes near the end of the lab. Worth 3 of the 10 points.

Why is there a checkpoint?

It shows what you can do alone

For most of the lab you may use hints and AI tools. The checkpoint is the one moment without them.

The policy is new

The questions use a new policy, made for you at that moment. An answer copied from a neighbour does not fit.

The instructor sees who needs help

Your answers arrive at once, so a common mistake can be explained before the next lab.

It is fair

Everyone gets the same kind of task, with their own policy.

What happens, step by step

1. The instructor opens it
Close your AI tools

→

2. oslab checkpoint lab6
Read the policy, answer four questions

→

3. The file task
New folder and data.txt, your modes

→

4. oslab check lab6
A fifth milestone must pass

Before it opens

The command says that the checkpoint is not open yet. Wait for the instructor.

While you work

Use your terminal and what you learned in this lab. No AI tools, and no help from a neighbour.

When you finish

oslab check lab6 shows whether the fifth milestone, checkpoint, passes. You can fix the modes and check again.

What the checkpoint asks

Part	What you do
Four questions	Read the new policy. Answer two octal modes, one yes or no, and one sentence.
The file task	Make the new folder and data.txt that the command prints, set your two modes, and save the stat output in checkpoint.txt.
The check	Run oslab check lab6. The fifth milestone, checkpoint, must show PASS.

The same skill as Core 1: policy sentence to octal modes, for a folder and a file.

DEBRIEF AND SUBMIT

Look back, then hand in

Ten minutes of debrief, ten to submit.

Debrief: look at your prediction again

student@server: ~/oslab-work/lab6

```
$ cat "$OSLAB_WORKSPACE/.records/lab6-predict.json"
$ oslab debrief lab6
```

oslab saved your prediction in a hidden folder, .records. It stays as it was, so the comparison shows which answers you confirmed and which you corrected.

DISCUSS

Class discussion

A file is readable by its mode, but you still got Permission denied. Where do you look?

Submit 1: your repository and the copy

student@server: ~

```
$ SUBMISSION_REPO="$HOME/os-gic-YOUR_ID/os-lab-YOUR_ID"
$ mkdir -p "$SUBMISSION_REPO/lab6/evidence"
$ cd "$OSLAB_WORKSPACE/lab6"
$ cp -- evidence/policy.txt evidence/traversal.txt "$SUBMISSION_REPO/lab6/evidence/"
```

Replace YOUR_ID with your own ID. The orange text is the part you change. Use the absolute path of your existing course repository, never a neighbour's.

Example: with the ID 0123, the first line becomes SUBMISSION_REPO="$HOME/os-gic-0123/os-lab-0123", and echo "$SUBMISSION_REPO" prints /home/student/os-gic-0123/os-lab-0123.

If you did Plus or Challenge, copy evidence/plus.txt the same way. The course website checks that these exact names are there. Check the final modes once more.

Submit 2: the final tree and Git

student@server: ~/os-gic-YOUR_ID/os-lab-YOUR_ID

```
$ tree
lab6
├── evidence
│   ├── policy.txt
│   └── traversal.txt
└── README.md

1 directory, 3 files
```

1. Write README.md from the lab's template, in your own words.
2. The prediction must have been written before you ran anything. Copying it into the README only records it.
3. git status --short, then add only your lab files, then commit and push.
4. No passwords, personal shell files, generated binaries or disk images.

Submit 3: commit and push

student@server: ~

```
$ cd ~/os-gic-YOUR_ID
$ git pull
$ git add os-lab-YOUR_ID/lab6
$ git commit -m "Lab 6"
$ git push
```

git pull comes first, in case you pushed from your own computer since.

Replace YOUR_ID with your own ID, as before.

How it is marked: 10 points

Evidence for the lab objectives	Points
Modes: oslab check lab6 passes the four Core milestones (private, your notice policy, the traversal test, policy.txt)	2
Tests: policy.txt and traversal.txt show the modes and the failure, and the README separates what you tested from what you reasoned	2
Prediction saved in time, and corrected with oslab debrief lab6	2
Live checkpoint: the answers and the new folder with the right modes	3
Complete submission: the required files in your repository, on time	1

Every row is marked from your oslab records and the files in your repository, so run oslab check lab6 until it shows what you expect. Your prediction earns its first point for being made, not for being right. Plus and Challenge are not needed for full marks.

Common mistakes

Mistake	What you see	Fix
Mode 644 on a file in a 700 folder	Others still cannot reach it	Check every parent of the path
Directory with r but no x	Names listed, file not opened	Add x for search
Running a data file to test read	Permission denied, and a wrong lesson	Use cat, and test -x for execute
Widening your home to test a peer	A private folder opened to everyone	Reason it, or ask for a prepared account
Typing BOX, or leaving the demo folder at 600	A folder named BOX, or a sealed folder in your evidence	Use your own box value, and chmod 700 after the test

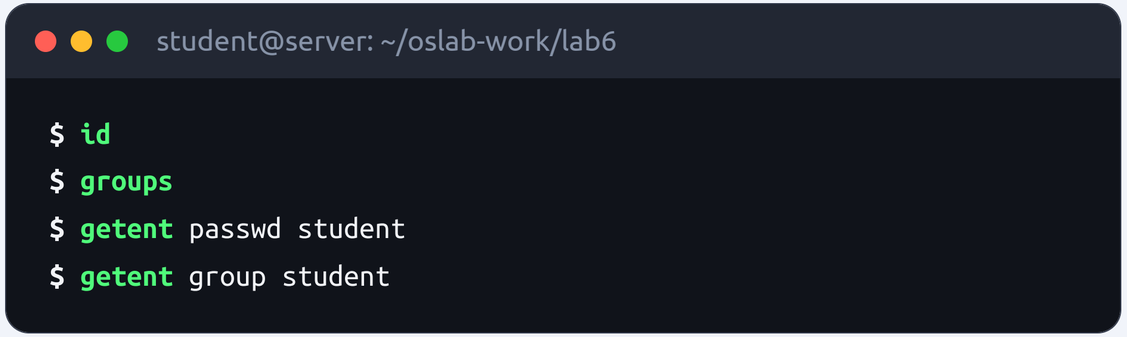
Accounts, ACLs and special bits

Optional extensions. They need prepared identities or a disposable virtual machine.

Lab 6

Accounts and groups

File	Holds
/etc/passwd	Accounts: name, uid, gid, home, shell
/etc/group	Groups and their members
/etc/shadow	Password hashes, readable by root only



Creating accounts and groups needs administrator rights. Do it only on a disposable virtual machine.

ACLs and special bits

ACL

ACLs

Extra rules for named users or groups, on top of the mode. getfacl shows them and setfacl changes them.

SPECIAL BITS

setuid and setgid

A program runs with its owner's or group's rights, not the caller's. On a directory, setgid makes new files inherit the group.

STICKY

Sticky bit

In a shared folder such as /tmp, only a file's owner may remove it, even if others can write the folder.

ls -l shows s and t in place of x, and chmod can set them with a fourth octal digit, for example 2755 or 1777.

Key takeaways

01

Three triples

Owner, group and others each get r, w and x. The first match decides.

02

Directory x is search

Without x on a folder you cannot reach what is in it, whatever the file's own mode says.

03

Check the whole path

The strictest folder between / and the file decides.

04

Observed or reasoned

Say which claims you tested and which you only worked out.

Next session

COMING UP

Lab 7: Bash scripting and server automation

Put commands in a script, pass it arguments, and make it safe to run.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 17, Protection.